

# Arquitecturas de Despliegue LLM

## Módulo 9 · Infraestructura y Despliegue

API Gateway, load balancing model-aware, autoscaling con KEDA, warm pools y arquitectura disaggregada.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales  
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

## Arquitecturas de Despliegue LLM

Un LLM en producción no es solo un proceso que corre en una GPU: es un sistema distribuido con múltiples componentes que deben diseñarse para la disponibilidad, la escalabilidad, y la eficiencia. La arquitectura de despliegue define cómo se estructura ese sistema: cómo se expone el modelo al mundo exterior (API Gateway), cómo se distribuye la carga entre múltiples instancias del modelo (load balancing), cómo se escala automáticamente ante picos de demanda (autoscaling), y cómo se garantiza la disponibilidad ante fallos de componentes individuales (high availability).

En 2025, el estándar emergente para despliegue de LLMs en producción es vLLM sobre Kubernetes, con el Kubernetes Gateway API Inference Extension (introducido en junio 2025) como plano de control para routing inteligente específico de LLMs. El proyecto llm-d (open source, 2025) integra vLLM + Kubernetes Inference Gateway para despliegues distribuidos de alto rendimiento, con 40% de reducción de latencia por token respecto al routing round-robin estándar.

## Los componentes de una arquitectura de despliegue LLM

### API Gateway: la entrada controlada al sistema

El API Gateway es el punto único de entrada para todas las peticiones al sistema LLM. Responsabilidades: autenticación y autorización (API keys, JWT, OAuth), rate limiting (límites de uso por cliente o por plan), routing (enviar la petición al backend correcto según el modelo solicitado o las características de la petición), logging de todas las peticiones (para monitorización, auditoría, y facturación), y gestión de versiones de la API. Herramientas estándar: Kong, NGINX, AWS API Gateway, Azure API Management. Para LLMs específicamente, el Kubernetes Gateway API Inference Extension proporciona routing model-aware (dirige peticiones al pod que tiene el KV cache del prefijo, reduciendo latencia).

### Load balancing: distribuir la carga entre instancias

En LLMs, el load balancing tradicional (round-robin) es subóptimo porque no considera el estado interno de los pods de inferencia (cuántas peticiones están en proceso, si tienen el KV cache de un prefijo en concreto). El load balancing model-aware considera: la utilización actual del KV cache de cada pod, el prefijo de la petición entrante (para enviarla al pod que ya tiene ese prefijo cacheado), y la prioridad de la petición (consultas interactivas vs batch). El Kubernetes Gateway API Inference Extension implementa esto, con mejoras de latencia de hasta 30% sobre round-robin.

### Autoscaling: adaptarse a la demanda

El autoscaling de pods de inferencia LLM tiene requisitos distintos al autoscaling de microservicios web: el tiempo de arranque de un pod de inferencia (cargar el modelo en VRAM) es de 1-5 minutos, mucho mayor que el de un microservicio web (segundos). Esto requiere

estrategias de warm pool (mantener pods pre-calentados) y scale-to-zero para periodos de no uso. KEDA (Kubernetes Event-Driven Autoscaler) permite escalar basándose en métricas personalizadas como la longitud de la cola de peticiones, el throughput de tokens, o la utilización de GPU.

### SECCIÓN 3 · CÓMO FUNCIONA

---

## El stack de despliegue estándar en Kubernetes

La arquitectura de referencia para LLMs en Kubernetes en 2025: capa de entrada (Kubernetes Gateway API Inference Extension o NGINX + rate limiting), capa de inferencia (Deployment de pods vLLM con GPU, HPA + KEDA para autoscaling), almacenamiento de modelos (shared storage como NFS o S3 para los pesos del modelo, evitando tener que descargar el modelo en cada arranque de pod), observabilidad (Prometheus + DCGM Exporter para métricas GPU, Grafana para dashboards, OpenTelemetry para tracing), y gestión de secretos (Kubernetes Secrets o HashiCorp Vault para API keys y credenciales).

La configuración del Deployment de vLLM en Kubernetes requiere: especificar las GPU necesarias (`resources.limits.nvidia.com/gpu: 1` para 1 GPU), configurar el node selector para nodos GPU, definir los liveness y readiness probes (/health endpoint de vLLM), y configurar los resource requests correctos para garantizar QoS. Las persistent volume claims (PVC) para almacenar los pesos del modelo reducen el tiempo de arranque de un pod de 5-10 minutos (descarga desde internet) a 1-2 minutos (carga desde NFS local).

### Alta disponibilidad y manejo de fallos de GPU

Las GPUs en data centers empresariales fallan al 5-10% anual. Una arquitectura de alta disponibilidad para LLMs requiere: múltiples réplicas del pod de inferencia (mínimo 2 para HA básico, 3+ para HA real con tolerancia a fallo de zona de disponibilidad), health checks con liveness y readiness probes que detectan fallos del proceso de inferencia, política de reinicio automático en caso de fallo (RestartPolicy: Always), y PodDisruptionBudget para garantizar que al menos N-1 réplicas están disponibles durante actualizaciones o disrupciones planificadas.

### SECCIÓN 4 · EJEMPLOS REALES

---

- Arquitectura de Salesforce Engineering para LLM interno: múltiples réplicas de Llama 3.1 8B con vLLM en Kubernetes, autoscaling con KEDA basado en longitud de cola, warm pool de 3 réplicas mínimas para eliminar cold starts en horas de mayor uso, NGINX como API Gateway con rate limiting por equipo. El SLA es 99.5% de disponibilidad y P99 < 3s.
- Arquitectura llm-d en producción (DigitalOcean + Kubernetes): modelo base en pods prefill (A100 para procesamiento del prompt) + pods decode (GPU más eficiente para generación de tokens). El Kubernetes Inference Gateway enruta peticiones al pool prefill primero y luego al decode, optimizando la utilización. Benchmark: 40% reducción de latencia por token respecto a arquitectura monolítica.
- Multi-LoRA serving en producción (SaaS con múltiples clientes): un único Deployment de vLLM con el modelo base Llama 3 8B sirve múltiples clientes con diferentes adaptadores LoRA. El API Gateway enruta cada petición con el adaptador correcto basándose en el API key del cliente. Un solo pod de GPU sirve 12 adaptadores LoRA distintos, reduciendo el

coste de infraestructura 12x respecto a una instancia por adaptador.

## SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

---

Error 1: Usar round-robin load balancing sin considerar el estado del KV cache. El round-robin envía peticiones al siguiente pod independientemente de si tiene el KV cache del prefijo de esa petición. El routing prefix-aware reduce el tiempo de prefill en 50-90% para peticiones con el mismo system prompt largo, una mejora enorme para sistemas RAG donde el contexto recuperado puede ser el mismo en múltiples consultas.

Error 2: No configurar warm pools para modelos grandes. Sin warm pool, el primer usuario que llega después de un periodo de inactividad experimenta 5-10 minutos de cold start mientras el pod descarga el modelo. Para sistemas de uso empresarial con SLAs, esto es inaceptable. Configura siempre un número mínimo de réplicas >0 en producción.

Error 3: No configurar los PodDisruptionBudgets. Sin PDB, una actualización del Deployment puede terminar todos los pods simultáneamente, produciendo una interrupción de servicio. El PDB garantiza que siempre hay al menos N-1 pods disponibles durante las disrupciones planificadas.

## SECCIÓN 6 · APLICACIÓN PRÁCTICA

---

Para desplegar vLLM en Kubernetes: crea un Deployment con la imagen nvidia/cuda base + vLLM instalado, especifica `resources.limits.nvidia.com/gpu: 1`, configura el comando de inicio como `python -m vllm.entrypoints.openai.api_server --model modelo --port 8000`. Crea un Service de tipo ClusterIP para el Deployment. Configura el Ingress o Gateway API para exponer el servicio externamente con TLS. Añade el HPA con la métrica de throughput de tokens de vLLM vía KEDA.

Para la gestión de los pesos del modelo: descarga el modelo desde Hugging Face una sola vez a un volumen persistente compartido (NFS o EFS en AWS). Configura el Deployment para montar ese volumen en `/models`. vLLM lee los pesos desde `/models/nombre_del_modelo` en lugar de descargarlos en cada arranque. Esto reduce el cold start de 5-10 minutos a 1-2 minutos y elimina el riesgo de descarga fallida en arranque.

## SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

---

- M8-T6 (vLLM y TGI): los servidores de inferencia son el componente central de la arquitectura de despliegue.
- M9-T3 (APIs de IA): el API Gateway es el componente de exposición de los servicios LLM.
- M9-T5 (Escalabilidad): el autoscaling con KEDA es el mecanismo de escalabilidad de la arquitectura.

## SECCIÓN 8 · RESUMEN EJECUTIVO

---

La arquitectura estándar de despliegue LLM en Kubernetes tiene: API Gateway (Kong/NGINX + rate limiting + routing model-aware), Deployment de vLLM con GPU (múltiples réplicas para HA), autoscaling con KEDA (basado en longitud de cola o throughput), warm pool (mínimo de réplicas > 0), PVC para pesos del modelo (reduce

cold start), observabilidad (Prometheus + Grafana + OpenTelemetry). La arquitectura disaggregada (prefill y decode en pods separados) reduce la latencia 40%. El proyecto llm-d proporciona Helm charts production-ready con todas estas best practices. PodDisruptionBudgets para HA durante actualizaciones.